

real-time-systems-course Guía 1 de Trabajos Prácticos

practical exercises and projects of the real-time systems course GUÍA 1

Desafío 1

Preguntas: ¿Todas las tareas se ejecutan? ¿Hay concurrencia entre tareas? ¿Por qué?. ¿Los tiempos de conmutación de los leds son precisos? (Medirlos de ser posible utilizando periodos de conmutación más pequeños: 20ms, 40ms, 60ms y 80ms)

Análisis:

Para este desafío realizamos 4 prototipos de tareas para dejar en claro una *Mala práctica* y no utilizar la posibilidad de pasarle parámetros a las mismas.

```
void vTaskLed1(void * pvParameters);
void vTaskLed2(void * pvParameters);
void vTaskLed3(void * pvParameters);
void vTaskLed4(void * pvParameters);

void vTaskLed1(void * pvParameters){
    uint32_t Delay = 200;

    while(1){
        HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_12); // Green

        HAL_Delay(Delay);
    }
}

int main(){
    xTaskCreate(vTaskLed1, "Task1", 100, NULL, 0, NULL);
    xTaskCreate(vTaskLed2, "Task2", 100, NULL, 0, NULL);
    xTaskCreate(vTaskLed3, "Task3", 100, NULL, 0, NULL);
    xTaskCreate(vTaskLed4, "Task4", 100, NULL, 0, NULL);
}
```

En cada una declaramos el valor de delay necesario cuando podríamos haberlo pasado por parámetro. Este programa utiliza el servicio `HAL_Delay()` que es una función que simula un loop requiriendo tiempo de procesamiento del micro hasta que termine el mismo. Lo que vemos como resultado de tener las 4 tareas creadas con la misma prioridad y tratando de correr generando una sensación de concurrencia pero esto es producto de la configuración de nuestro micro precisamente al Time Slicing: Si hay dos tareas de la misma prioridad listas, el scheduler alternará entre ellas en cada iteración del Tick Interrupt (Round Robin).

Los tiempos que vemos no son precisos ya que al tener todas la misma prioridad el micro le da una porción de procesamiento a cada una y ninguna termina su loop al tiempo que determinamos.

Desafío 2

Preguntas: ¿Qué diferencias hay en las transiciones de tareas respecto al caso del Desafío 1? (De ser posible medir con fines comparativos utilizando los mismos periodos de conmutación del caso anterior).

Análisis:

Para este desafío empezamos a parametrizar las configuraciones y los parámetros de los LEDS (en el futuro serán otros periféricos) y empezamos a definir tareas más generales para solo tener instancias de la misma.

ESTRUCTURA DE PARÁMETROS DE LOS LEDS

```
typedef struct {
    GPIO_TypeDef* GPIO_puerto;
    uint16_t GPIO_pin;
    uint32_t delay;
}Led_Param_t;
```

TAREA PARAMETRIZADA

```
void vTareaParpadeo(void * pvParameters){
    Led_Param_t *pxParam = (Led_Param_t *) pvParameters;

    while(1){

        HAL_GPIO_TogglePin(pxParam->GPIO_puerto, pxParam->GPIO_pin);

        vTaskDelay(pdMS_TO_TICKS(pxParam->delay));
    }
}
```

En el Desafío 1 las tareas pasaban únicamente de Ready a Running consumiendo el 100% de la CPU y desperdiciando recursos. Mientras que en el Desafío 2 las tareas ahora pasan por Running, Blocked y Ready. Tardan apenas unos microsegundos en hacer el Toggle del pin y calcular el nuevo vTaskDelay, liberando la CPU. Ahora, durante el 99.9% del tiempo, las 4 tareas están en estado Blocked.

Con un osciloscopio podríamos ver los tiempos de onda y calcular el tiempo que demora; en teoría los tiempos tendrían una exactitud muy grande acercándose casi a los tiempos que **determinamos**.

Desafío 3

Preguntas: Probar con diferentes tiempos para vTaskDelay(). ¿Qué efecto se observa a medida que ese tiempo aumenta? ¿Qué tiempo satisface el deadline impuesto en la consigna?

Análisis

Para este desafío empezamos a introducir nuevos periféricos y la forma de poder interactuar con ellos. La manera que vamos a ver es la de Polling pero para evitar el starvation la tarea de alta prioridad vPollingButton() debe liberar voluntariamente la CPU. Al llamar a vTaskDelay(), la tarea del botón pasa al estado Blocked. Mientras está bloqueada, consume 0% de CPU, permitiendo que la tarea del LED (baja prioridad) se ejecute.

// TAREA ENCARGADA DE HACER POLLING SIN STARVATION

```
void vPollingButton(void * pvParameters){
    GPIO_PinState status_button;
    Led_Param_t *pxParam = (Led_Param_t *) pvParameters;
```

```

while(1){
    status_button = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);

    if(status_button == GPIO_PIN_SET){
        HAL_GPIO_WritePin(pxParam->GPIO_puerto, pxParam->GPIO_pin, GPIO_PIN_SET);
    }else{
        HAL_GPIO_WritePin(pxParam->GPIO_puerto, pxParam->GPIO_pin, GPIO_PIN_RESET);
    }

    vTaskDelay(pdMS_TO_TICKS(10));
}
}

```

Si variamos los tiempos de lectura del botón el LED parpadea normalmente, pero el botón se volverá “lento”. Si presionamos y soltamos el botón muy rápido (en menos de 500ms), la tarea del botón estará en estado Blocked y ni siquiera leerá que presionamos el botón y no sucederá nada, el estado del led no cambiará.

Para respetar el tiempo de respuesta del LED 2 y que no supere los 10 ms, la tarea debe revisar el botón al menos cada 10 milisegundos, por ende debemos usar `vTaskDelay(pdMS_TO_TICKS(10))`. La tarea se bloquea por 10ms y lee el botón, actualiza el LED y vuelve a bloquearse.

Desafío 4

Preguntas: Intuitivamente determinar el tiempo (?) argumento de los servicios `xTaskDelay()` y `xTaskDelayUntil()` para cumplir con los tiempos impuestos en la consigna. ¿Ambos argumentos son iguales? ¿Qué efecto se observa si lo fueran? ¿Por qué?

Análisis:

Para este desafío vamos a introducir el análisis de los tiempos de delay y de cuándo se inicia el conteo de los mismos. Para ello usaremos dos tareas que parecen similares pero son conceptualmente diferentes. Mientras que `xTaskDelay()` cuenta el tiempo desde que es llamada sin importarle lo que sucede en el medio (tiempos de lectura de periféricos, procesamiento de datos, etc), el servicio `xTaskDelayUntil()` resuelve el problema especificando la hora exacta en la que la tarea debe desbloquearse, sin importar cuánto tardó en ejecutarse el código previo (siempre y cuando el código tarde menos que el periodo total).

```

void vTareaParpadeo_Delay(void * pvParameters){
    Led_Param_t *pxParam = (Led_Param_t *) pvParameters;

    while(1){

        HAL_GPIO_TogglePin(pxParam->GPIO_puerto, pxParam->GPIO_pin);

        HAL_Delay(100);

        vTaskDelay(pxParam->delay);
    }
}

void vTareaParpadeo_DelayUntil(void * pvParameters){
    Led_Param_t *pxParam = (Led_Param_t *) pvParameters;

    TickType_t xLastWakeTime = xTaskGetTickCount();
}

```

```

while(1){

    HAL_GPIO_TogglePin(pxParam->GPIO_puerto, pxParam->GPIO_pin);

    HAL_Delay(100);

    vTaskDelayUntil(&xLastWakeTime, pxParam->delay);

}
}
void vPollingButton(void * pvParameters){
    GPIO_PinState status_button;
    Led_Param_t *pxParam = (Led_Param_t *) pvParameters;

    while(1){
        status_button = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);

        if(status_button == GPIO_PIN_SET){
            HAL_GPIO_WritePin(pxParam->GPIO_puerto, pxParam->GPIO_pin, GPIO_PIN_SET);
        }else{
            HAL_GPIO_WritePin(pxParam->GPIO_puerto, pxParam->GPIO_pin, GPIO_PIN_RESET);
        }

        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

```

Para lograr el comportamiento esperado sabemos que uno de los servicios no va a tener en cuenta el tiempo que se pasa en el servicio `HAL_Delay()` mientras que el otro sí, por ende hay que tener en cuenta esta diferencia entre tiempo relativo y tiempo absoluto. Intuitivamente la diferencia es de 100, por ende la tarea que utilice `xTaskDelayUntil()` va a tener en cuenta esto y por ende le tenemos que restar esos 100, quedando `Led_Param_t Led1_delay = {GPIOD, GPIO_PIN_12, 400}`; `Led_Param_t Led2_delay = {GPIOD, GPIO_PIN_13, 500}`; respectivamente. Si fueran iguales veríamos un desfase en los LEDS porque `vTaskDelay` empieza a contar los Ticks de bloqueo después de que se terminó de procesar el `HAL_Delay(100)`.

Desafío 5

Preguntas: ¿Qué ocurre cuando la prioridad de la tarea A se restablece? ¿Qué servicio se utilizó para hacer el conteo de los 3 segundos? ¿Se logró el comportamiento descrito en el enunciado en el primer intento? De no ser así, ¿por qué?

Análisis

En este desafío nos introducimos a cómo el scheduler (y su configuración) determinan qué tareas se van a procesar teniendo en cuenta principalmente *la prioridad*.

```

int main(){
    xTaskCreate(vTareaParpadeo_Delay, "Tarea_LedA", 100, (void *) &LedA_delay, 1, &xTarea_LedA_handle);
    xTaskCreate(vTareaParpadeo_Delay, "Tarea_LedB", 100, (void *) &LedB_delay, 1, NULL);
    xTaskCreate(vPollingButton_Priority, "Button_change", 100, NULL, configMAX_PRIORITIES - 1, NULL);
}
void vTareaParpadeo_Delay(void * pvParameters){
    Led_Param_t *pxParam = (Led_Param_t *) pvParameters;

```

```

while(1){

    HAL_GPIO_TogglePin(pxParam->GPIO_puerto, pxParam->GPIO_pin);

    HAL_Delay(pxParam->delay);

}
}
void vPollingButton_Priority(void * pvParameters){
    GPIO_PinState status_button;

    while(1){
        status_button = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);

        if(status_button == GPIO_PIN_SET){

            TickType_t xLastWakeTime = xTaskGetTickCount();

            UBaseType_t priority = uxTaskPriorityGet(xTarea_LedA_handle) + 1;
            vTaskPrioritySet(xTarea_LedA_handle, priority);

            vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(3000));

            // Deberíamos tener una condición if tal que priority > 0 siempre en este punto
            vTaskPrioritySet(xTarea_LedA_handle, priority - 1);

        }

        vTaskDelay(pdMS_TO_TICKS(10));

    }
}

```

Pudimos lograr el comportamiento esperado ya que utilizamos en el lugar correcto los servicios `uxTaskPriorityGet()` y `vTaskPrioritySet()`; los mismos que son los encargados de leer y setear las prioridades de las tareas deben ser utilizados en los servicios que generan los eventos como en este caso el Polling del botón. En el momento en que `vTaskPrioritySet(NULL, uxPrioridadOriginal)` se ejecuta, la Tarea A vuelve a Prioridad 1 y como la Tarea B (Prioridad 1) probablemente llevaba casi 3 segundos intentando ejecutarse (estaba en estado Ready esperando CPU), el Scheduler en el siguiente Tick le devuelve el control a la Tarea B. Ambas vuelven a compartir la CPU y ambos LEDs retoman su parpadeo de 300ms.

Desafío 6

Preguntas: Reemplazar el polling por un mecanismo de espera pasiva. Proponer una forma de visualizar dinámicamente (algo cambia todo el tiempo) que la tarea en cuestión efectivamente no está consumiendo tiempo del procesador. No está permitido crear nuevas tareas.

Análisis

En este desafío nos introducimos en las interrupciones para solucionar el caso de estar todo el tiempo consultando al botón si fue activado. La solución es el Procesamiento Diferido:

- Ocurre la interrupción de hardware (el pulsador en EXTI0).

- La ISR simplemente avisa a una tarea que el evento ocurrió (desbloqueándola) y termina inmediatamente. Para enviar este “aviso” desde la ISR a la Tarea, utilizamos un Semáforo Binario.
- La Tarea, que estaba en estado Blocked, consumiendo 0 CPU, se despierta y realiza el trabajo pesado.

```

Led_Param_t param_Led3 = {GPIOD, GPIO_PIN_14, 50};
Led_Param_t param_Led4 = {GPIOD, GPIO_PIN_15, 0}; // Para la tarea mínima de scheduler (Tarea Idle)

void vBlinkyLed_Deferred(void * pvParameters){
    Led_Param_t *pxParam = (Led_Param_t *) pvParameters;
    while(1){

        if(xSemaphoreTake(semaphored_button, portMAX_DELAY) == pdPASS){

            HAL_GPIO_TogglePin(pxParam->GPIO_puerto, pxParam->GPIO_pin);

        }
        // vTaskDelay(pdMS_TO_TICKS(pxParam->delay));
    }
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){

    if(GPIO_Pin == GPIO_PIN_0){
        BaseType_t xHigherPriorityTaskWoken = pdFALSE;

        xSemaphoreGiveFromISR(semaphored_button, &xHigherPriorityTaskWoken);

        /* 5. Si xHigherPriorityTaskWoken se puso en pdTRUE, forzamos un cambio de contexto
        * para que al salir de la interrupción entremos directo a la tarea del botón. */
        //portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }

}

void vApplicationIdleHook(void)
{
    HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_15); // LED4
    HAL_Delay(200);
}

if((semaphore_L2_L3 != NULL) && (semaphore_L3_L4 != NULL) && (semaphore_L4_L2 != NULL)){

    xTaskCreate(vBlinkyLed2_Semaphore, "Led 2", 100, (void *) &param_Led2_Led3, 0, NULL);
    xTaskCreate(vBlinkyLed3_Semaphore, "Led 3", 100, (void *) &param_Led3_Led4, 0, NULL);
    xTaskCreate(vBlinkyLed4_Semaphore, "Led 4", 100, (void *) &param_Led4_Led2, 0, NULL);

}

xSemaphoreGive(semaphore_L4_L2); // Iniciar la secuencia Para comprobar el comportamiento esperado
utilizamos la función mínima que utiliza FreeRTOS (el scheduler) para cuando no hay nada para hacer. Esta
es la void vApplicationIdleHook(void). Activaremos el Idle Hook y haremos que parpadee un LED. Este
seguirá parpadeando por más que nosotros toquemos muchas veces el botón o no lo toquemos porque es la
tarea que siempre ejecuta el scheduler.

```

Desafío 7

Preguntas: ¿Qué se debe cambiar para alterar el orden de la secuencia? ¿Es posible implementar una solución a este problema escribiendo una única función?

Análisis

En este desafío vamos a ver cómo varias tareas se pueden sincronizar entre ellas gracias a los *semáforos*. Vamos a utilizar la idea de un círculo y cada tarea toma un semáforo para luego desbloquear el de la tarea siguiente.

```
int main(){
    if((semaphore_L2_L3 != NULL) && (semaphore_L3_L4 != NULL) && (semaphore_L4_L2 != NULL)){

        xTaskCreate(vBlinkyLed2_Semaphore, "Led 2", 100, (void *) &param_Led2_Led3, 0, NULL);
        xTaskCreate(vBlinkyLed3_Semaphore, "Led 3", 100, (void *) &param_Led3_Led4, 0, NULL);
        xTaskCreate(vBlinkyLed4_Semaphore, "Led 4", 100, (void *) &param_Led4_Led2, 0, NULL);
    }
    xSemaphoreGive(semaphore_L4_L2); // Iniciar la secuencia
}

void vBlinkyLed2_Semaphore(void * pvParameters){
    Led_Param_t *pxParam = (Led_Param_t *) pvParameters;

    while(1){

        if(xSemaphoreTake(semaphore_L4_L2, portMAX_DELAY) == pdPASS){

            HAL_GPIO_TogglePin(pxParam->GPIO_puerto_reset, pxParam->GPIO_pin_reset);
            HAL_GPIO_TogglePin(pxParam->GPIO_puerto_set, pxParam->GPIO_pin_set);
            HAL_Delay(pxParam->delay);

            xSemaphoreGive(semaphore_L2_L3);
        }
        // vTaskDelay(pdMS_TO_TICKS(pxParam->delay));
    }
}
```

Mientras realizamos el desafío notamos cómo las tareas son muy similares entre sí y propusimos la idea de ampliarla de manera tal que tengamos todos los parámetros necesarios para realizar lo mismo que hicimos en 3 tareas independientes. Para la tarea general `vBlinkyLed_Semaphore()` tendríamos todos los datos necesarios y sería simplemente extraer los datos del parámetro `*pvParameters`.

```
typedef struct {
    SemaphoreHandle_t xSemTomar;    // Semáforo que debe esperar
    SemaphoreHandle_t xSemDar;      // Semáforo que debe liberar

    GPIO_TypeDef* GPIO_puerto_set; // Parámetros del LED a encender
    uint16_t GPIO_pin_set;
    uint32_t delay;

    GPIO_TypeDef* GPIO_puerto_reset; // Parámetros del LED a apagar
    uint16_t GPIO_pin_reset;
} ParametrosSecuencia_t;
```

Desafío 8

Preguntas: Comprender el acceso exclusivo a recursos compartidos. ¿Se debe usar servicios bloqueantes (vTaskDelay()) o no-bloqueantes (HAL Delay()) para contar los tiempos asociados a una secuencia? ¿Por qué? Reescriba el ejercicio utilizando el otro servicio de Delay al propuesto originalmente y compare su comportamiento.

Análisis

En este desafío vamos a utilizar un mutex para sincronizar dos secuencias diferentes de LEDS. Ya que es crítico el uso de los LEDS todo queda bajo el control del mutex y la otra tarea que quiera hacer uso de ellos deberá esperar el recurso.

```
const Led_Param_t leds[4] = {{GPIOD, GPIO_PIN_12, 500}, {GPIOD, GPIO_PIN_13, 500},
                             {GPIOD, GPIO_PIN_14, 600}, {GPIOD, GPIO_PIN_15, 800}};

int main(){

    Semaphored_Mutex_LEDS = xSemaphoreCreateMutex();

    if(Semaphored_Mutex_LEDS != NULL){

        xTaskCreate(vBlinky4Leds, "4 Leds ", 100, (void *) &leds, 0, NULL);
        xTaskCreate(vBlinkyLedsSecuence, "4 Leds ", 100, (void *) &leds, 0, NULL);

    }
}

// TAREAS QUE SE SINCRONIZAN CON EL MUTEX

void vBlinkyLedsSecuence(void * pvParameters){
    Led_Param_t *pxParam = (Led_Param_t *) pvParameters;

    while(1){

        xSemaphoreTake(Semaphored_Mutex_LEDS, portMAX_DELAY); {

            for (int i = 0; i < 4; i++) {
                HAL_GPIO_WritePin(pxParam[i].GPIO_puerto, pxParam[i].GPIO_pin, GPIO_PIN_SET);
                HAL_Delay(200);
                //vTaskDelay(pdMS_TO_TICKS(200));

                HAL_GPIO_WritePin(pxParam[i].GPIO_puerto, pxParam[i].GPIO_pin, GPIO_PIN_RESET);
                HAL_Delay(200);
                //vTaskDelay(pdMS_TO_TICKS(200));
            }

        }

        xSemaphoreGive(Semaphored_Mutex_LEDS);
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}

void vBlinky4Leds(void * pvParameters){
    Led_Param_t *pxParam = (Led_Param_t *) pvParameters;
```



```

while(1){

    xSemaphoreTake(Semaphored_Mutex_LEDS, portMAX_DELAY); {

        for (int i = 0; i < 4; i++) {
            HAL_GPIO_WritePin(pxParam[0].GPIO_puerto, pxParam[0].GPIO_pin, GPIO_PIN_SET);
            HAL_GPIO_WritePin(pxParam[1].GPIO_puerto, pxParam[1].GPIO_pin, GPIO_PIN_SET);
            HAL_GPIO_WritePin(pxParam[2].GPIO_puerto, pxParam[2].GPIO_pin, GPIO_PIN_SET);
            HAL_GPIO_WritePin(pxParam[3].GPIO_puerto, pxParam[3].GPIO_pin, GPIO_PIN_SET);
            HAL_Delay(200);
            //vTaskDelay(pdMS_TO_TICKS(200));

            HAL_GPIO_WritePin(pxParam[0].GPIO_puerto, pxParam[0].GPIO_pin, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(pxParam[1].GPIO_puerto, pxParam[1].GPIO_pin, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(pxParam[2].GPIO_puerto, pxParam[2].GPIO_pin, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(pxParam[3].GPIO_puerto, pxParam[3].GPIO_pin, GPIO_PIN_RESET);
            HAL_Delay(200);
            //vTaskDelay(pdMS_TO_TICKS(200));
        }
    }
    xSemaphoreGive(Semaphored_Mutex_LEDS);
    vTaskDelay(pdMS_TO_TICKS(50));
}
}

```

Tanto `vTaskDelay()` como `HAL_Delay()` tienen un efecto visual similar pero interna y conceptualmente son MUY diferentes. - Con `HAL_Delay()`: Utilizamos el 100% de la CPU y retenemos el Mutex. La otra tarea jamás se ejecuta hasta que soltemos el Mutex. Funciona visualmente, pero es catastrófico para la eficiencia energética y el determinismo general.

- Con `vTaskDelay()`: La tarea libera la CPU, permitiendo que otras tareas se ejecuten. Sin embargo, retenemos el Mutex mientras está bloqueada. La otra tarea intentará tomar el Mutex, fallará y se bloqueará. Esto es más eficiente a nivel de CPU permitiendo que el sistema haga otras cosas mientras estamos utilizando el recurso crítico.

Desafío 9

Preguntas: ¿Qué consecuencias tiene para el sistema que el estado del pulsador se determine por Polling o por interrupción? ¿Implementar la que ud. considere la solución que genera el mejor tiempo de respuesta? ¿Está seguro?

Análisis:

En este desafío vamos a introducir el estado que nos faltaba por usar: el de *Suspended*. Vamos a tener una tarea con mayor prioridad que va a suspender a otra luego de que un periférico realice una interrupción.

```

TaskHandle_t xHandleTaskA = NULL;
SemaphoreHandle_t Semaphored_button = NULL;

void vBlinkyLedsSequence(void * pvParameters){
    Led_Param_t *pxParam = (Led_Param_t *) pvParameters;

```

```

while(1){
    for (int i = 0; i < 4; i++) {
        HAL_GPIO_WritePin(pxParam[i].GPIO_puerto, pxParam[i].GPIO_pin, GPIO_PIN_SET);
        HAL_Delay(200);
        //vTaskDelay(pdMS_TO_TICKS(200));

        HAL_GPIO_WritePin(pxParam[i].GPIO_puerto, pxParam[i].GPIO_pin, GPIO_PIN_RESET);
        HAL_Delay(200);
        //vTaskDelay(pdMS_TO_TICKS(200));

    }
    vTaskDelay(pdMS_TO_TICKS(50));
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){

    if(GPIO_Pin == GPIO_PIN_0){
        BaseType_t xHigherPriorityTaskWoken = pdFALSE;

        xSemaphoreGiveFromISR(Semaphored_button, &xHigherPriorityTaskWoken);

        /* 5. Si xHigherPriorityTaskWoken se puso en pdTRUE, forzamos un cambio de contexto
        * para que al salir de la interrupción entremos directo a la tarea del botón. */
        //portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}

void vSuspension_Task(void * pvParameters){
    uint16_t isSuspended = 0;

    while(1){
        if(xSemaphoreTake(Semaphored_button, portMAX_DELAY)){
            if (isSuspended){
                vTaskResume(xHandleTaskA);
                isSuspended = 0;
            } else {
                vTaskSuspend(xHandleTaskA);
                isSuspended = 1;
            }
            vTaskDelay(pdMS_TO_TICKS(50));
            xSemaphoreTake(Semaphored_button, 0);
        }
    }
}

```

Realizamos la versión con interrupción del botón ya que:

- Polling:

Si la Tarea B hace un HAL_GPIO_ReadPin continuamente con un vTaskDelay(50ms) para no monopolizar la CPU, el tiempo de respuesta en el peor de los casos será de 50 ms.
Si no usa delay (espera activa), tendrá una respuesta casi instantánea,

pero generará Starvation (Inanición), arruinando el sistema.

- Interrupción (EXTI):

El hardware (botón en este caso) interrumpe el flujo de nuestro programa y mejorando el tiempo de respuesta ya que la rutina de interrupción de ese periférico libera un semáforo binario que habilita a la tarea vigilante que realiza la lógica para la suspensión.

El resultado es ver parpadear los LEDs. Al pulsar el botón, la ejecución de la Tarea de LEDS se detendrá en seco. Si los LEDs estaban encendidos, se quedarán encendidos. Si estaban apagados, apagados. Una segunda pulsación reanudará el ciclo exactamente donde se quedó.

Desafío 10

Preguntas: ¿Hay alguna forma de exteriorizar (mediante algún evento visible) que esa tarea efectivamente se ha eliminado y no se encuentra más disponible para su ejecución?

Análisis:

Finalmente en este desafío vemos cómo es posible eliminar de la memoria las tareas que instanciamos ya que en algunos casos queremos que tengan una vida útil y no acaparen recursos de memoria.

```
void vBlinkyLedsSequence(void * pvParameters){
    Led_Param_t *pxParam = (Led_Param_t *) pvParameters;
    TickType_t LastWakeTime;
    LastWakeTime = xTaskGetTickCount();
    while (1){
        for (int i = 0; i < 10; i++) {
            if (xSemaphoreTake(Semaphore_LEDS, portMAX_DELAY)) {
                HAL_GPIO_WritePin(pxParam[0].GPIO_puerto, pxParam[0].GPIO_pin, GPIO_PIN_SET);
                HAL_GPIO_WritePin(pxParam[1].GPIO_puerto, pxParam[1].GPIO_pin, GPIO_PIN_SET);
                HAL_GPIO_WritePin(pxParam[2].GPIO_puerto, pxParam[2].GPIO_pin, GPIO_PIN_SET);
                HAL_GPIO_WritePin(pxParam[3].GPIO_puerto, pxParam[3].GPIO_pin, GPIO_PIN_SET);
                xSemaphoreGive(Semaphore_LEDS);
                vTaskDelayUntil(&LastWakeTime, pdMS_TO_TICKS(500));
            }
            if (xSemaphoreTake(Semaphore_LEDS, portMAX_DELAY)) {
                HAL_GPIO_WritePin(pxParam[0].GPIO_puerto, pxParam[0].GPIO_pin, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(pxParam[1].GPIO_puerto, pxParam[1].GPIO_pin, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(pxParam[2].GPIO_puerto, pxParam[2].GPIO_pin, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(pxParam[3].GPIO_puerto, pxParam[3].GPIO_pin, GPIO_PIN_RESET);
                xSemaphoreGive(Semaphore_LEDS);
                vTaskDelayUntil(&LastWakeTime, pdMS_TO_TICKS(500));
            }
        }

        HAL_GPIO_WritePin(pxParam[3].GPIO_puerto, pxParam[3].GPIO_pin, GPIO_PIN_SET);
        vTaskDelete(NULL);
    }
}

void vApplicationIdleHook(void)
```

```

{
    uint16_t totalTasks = 0;
    totalTasks = uxTaskGetNumberOfTasks();

    if(totalTasks == 3){
        HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_12);
    }
    if(totalTasks == 4){
        HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_13);
    }
}

```

Para poder verificar que efectivamente se eliminó la tarea (no está más en nuestro sistema de tareas disponibles para el scheduler) usamos nuestra tarea *Idle* para que nos muestre con leds cuántas tareas quedan. En nuestro caso siempre tenemos 3 tareas disponibles (lo averiguamos con una investigación profunda del manual) y si le sumamos la tarea del desafío tendríamos 4. Entonces podemos ver en ambos casos en qué etapa estamos; si durante el bloqueo de la tarea `vBlinkyLedsSecuence` vemos el LED del pin 13 es porque todavía no se eliminó. Luego de la secuencia deberíamos ver cómo se prende solo el LED del pin 12.